Less Tests More Power

Samuel van Nimwegen
University of Antwerp
s0225763

29 September 2026

Master's thesis in the LORE research group
Promotor: Prof. Serge Demeyer. Daily supervisor: Tolgahan Bardakci.

1 Thesis

1.1 Context

A REST API test suite sends HTTP requests to a running service and checks the answers with assertions, which I call checks. Many tests repeat the same setup, such as creating a user and logging in, before they check one small thing. Removing tests cuts cost but also removes checks. I study two moves on top of removing tests. Tests can share requests, so one setup serves several checks. And inputs can move to the edges of what the service accepts, such as the longest allowed name [1]. My idea is that sharing saves cost and that boundary values win back faults that removing tests loses. By power I mean the share of planted faults that a suite detects.

1.2 Problem Statement

This thesis asks one main question. If I keep a fixed share of the tests of an existing REST API suite, how much can I cut its requests and resets (restores of a clean database) while it keeps its black box coverage and still finds the faults of the full suite? Black box coverage is my main metric. It counts which parts of the API the tests reach from the outside, such as operations, parameters and status codes [9]. Three research questions follow.

RQ1. What share of the requests and resets can tests avoid by sharing setup, when each kept check must still run as the same user, on the same resource and after the same earlier steps?

RQ2. With the same shared requests, does moving inputs to boundary values change the faults found, compared with random valid values, and which faults does it gain or lose?

RQ3. How long does the whole reduction take, does a graph that splits the tests into independent parts make the search faster, and after how many later runs does it pay off?

1.3 Relevance and Challenge

A study of five REST testing tools calls running every test on every release costly, and knows of no minimization approach for REST tests [2]. Removing tests is risky, because reduced suites lost the failure in up to 52.2 percent of the failed builds of 32 Java projects [3]. Testler [4], STICCER [5] and DJPlus [6] trim unit tests, merge database tests or build minimal suites from graph models, but none reduces existing tests that call a running service. In a literature search in September 2026 I found no work that reduces existing REST tests by sharing requests and boundary inputs.

The problem is hard because two requests that look the same differ when another user sends them or when they touch another resource. A careless merge lets a check pass for the wrong reason. A changed input can find a new fault and hide an old one, so reduced suites must run against faulty copies of the service. Resets are the main cost. For the 63 Python tests below they took 102 seconds, against 21 seconds for all 185 requests.

Before writing this scope I already ran a small sample of experiments on the Python service. The results were promising, but the sample is too small to draw conclusions. I describe them in more detail in Section 3.

2 Background

Over the past few years I have gained practical experience in REST API development and in software engineering in general, through projects at university and through my job. For the last two years I worked half time at TrackBox, where I did both software engineering and cloud engineering with microservices, Kubernetes and REST APIs. I really like working with REST APIs and with testing, so I was a big fan of this thesis topic. In the Software Testing course I studied boundary value testing, which this thesis builds on.

3 Proposed Solution Approach

The tool I build runs an existing suite once and stores its requests, inputs, answers, checks, users and resources in a graph that records the run. It then works in four steps.

- **Safe sharing:** Two tests share requests only when they open with the same requests, sent by the same user on the same resource, and each then adds at most one read or rejected request.
- **Boundary values:** The tool moves inputs to the edges of what the service accepts, from declared length limits in Python and values such as 0, 1 and the empty string in Java. A value used by several requests changes in all of them. With the original checks kept, this finds faults that reject valid values, not faults that accept invalid ones.
- **Replay:** The tool replays every proposal with the original checks unchanged. I also test this step on merges that I know are wrong.
- **Selection:** For a budget, the share of original tests I keep, an integer program picks tests, shared requests and boundary values that reach the most black box coverage, such as operations and statuses, and then the fewest resets and requests. Coverage is the main metric, and the faults show whether keeping it is enough. A shared

group runs after one reset and counts as all its tests, so a budget of 100 percent tests sharing alone. A second graph links tests that share coverage, checks or requests. I solve the program as a whole and once per separate part of that graph.

Since August I have built a graph builder and replay runner in Python and a harness for the Java tests. The Java pipeline runs end to end, but the automatic Python reduction has not run yet. I write the code with AI coding assistants and check every result against the raw logs.

With these tools I ran the small sample of experiments mentioned in Section 1.3. With the fixed inputs of my replay, my sharing rule allows at most 28 of the 185 requests of the 63 Python tests to be shared. This is an upper limit, because I have not yet replayed the shared groups. I also built one merged test by hand that replaces five original tests. It sends 8 requests instead of 23 and needs one reset instead of five. I then planted 48 faults by moving each of the 24 declared length limits of the service up or down by one. With values on the limits, the merged test caught 8 of them, all where a limit became stricter. The five original tests caught none. I chose the values of the merged test from the same limits that the faults change, so this is only a first sign.

4 Evaluation Metrics

I use two subjects. Restful Booker is a Node.js API with 110 Java tests from Nooyens et al. [7]. Only 35 pass on my copy and each sends one request, so it shows little about sharing. The FastAPI service of RESTestBench [8] comes with 63 Python tests, and I answer RQ1 on it.

I will compare random, greedy and optimal selection of whole tests with three variants of my method (sharing alone, sharing then boundary values, both in one search) and a control with random valid values, keeping 100, 75, 50 and 25 percent of the passing tests. Before I test any reduced suite I fix the faults. These are mutants, which are small planted code changes: all 141 RESTestBench mutants and about 50 Cosmic Ray mutants for Python, and about 50 Stryker mutants for Node.js.

4.1 Quantitative Metrics

- **Black box coverage (main metric):** Which operations, parameters and status codes the reduced suite still reaches, next to the full suite [9].
- **Requests and resets:** How many the reduced suite needs next to the full suite (RQ1).
- **Faults found:** The share of faults detected, per fault family and budget (RQ2).
- **Run time:** The time to run the reduced suite and to compute it, whole and split, as the median and worst case of repeated runs (RQ3).
- **Payback:** After how many later runs the reduction has paid back its cost (RQ3).

4.2 Qualitative Metrics

- **Fault changes:** Which faults only one of the two suites finds, and why (RQ2).
- **Sharing blockers:** Which test patterns stop tests from sharing requests (RQ1).

5 Rough Plan

Phase: Approval and Java check
Month: October
Description: I ask for approval of this scope and register in SISA. By 31 October I sort the 28 failing Java tests that the package notes do not explain.

Phase: Safe sharing
Month: November
Description: I extend safe sharing to the Python tests. By 15 November it must save at least 14 of the at most 28 requests that my sharing rule allows, while every kept check gives the same answer. If not, I report which test patterns block it.

Phase: Implementation
Month: December, January
Description: I add boundary values, resets in the search and the split solve for Python, and I build the mutant pipeline. By 31 January both subjects have a reduced suite and frozen faults.

Phase: Evaluation
Month: February, March
Description: I run the full evaluation. March is also buffer.

Phase: Writing
Month: April, May
Description: I send a full draft to my daily supervisor by the end of April. Prof. Demeyer reads it in May, and by early May we choose June or August.

Phase: Submission
Month: June
Description: I submit and defend. August is the fallback.

References

[1] Tolgahan Bardakci et al. Test Amplification for REST APIs: Using "Out-of-the-Box" Large Language Models. IEEE Software 42(4), 2025. doi:10.1109/MS.2025.3559664

[2] Hassan Sartaj et al. REST API Testing in DevOps: A Study on an Evolving Healthcare IoT Application. ACM TOSEM 35(6), 2026. doi:10.1145/3765744

[3] August Shi et al. Evaluating Test-Suite Reduction in Real Software Evolution. ISSTA, 2018. doi:10.1145/3213846.3213875

[4] Arash Vahabzadeh et al. Fine-Grained Test Minimization. ICSE, 2018. doi:10.1145/3180155.3180203

[5] Abdullah Alsharif et al. STICCER: Fast and Effective Database Test Suite Reduction Through Merging of Similar Test Cases. ICST, 2020. doi:10.1109/ICST46399.2020.00031

[6] Yavuz Köroğlu et al. DJPlus: Generating Minimal Test Suites for Strong Coverage Criteria in Graph Models. arXiv:2609.08953

[7] Robbe Nooyens et al. Test Amplification for REST APIs via Single and Multi-Agent LLM Systems. Testing Software and Systems, LNCS, 2025. arXiv:2504.08113

[8] Leon Kogler et al. RESTestBench: A Benchmark for Evaluating the Effectiveness of LLM-Generated REST API Test Cases from NL Requirements. EASE, 2026. arXiv:2604.25862

[9] Alberto Martin-Lopez, Sergio Segura and Antonio Ruiz-Cortés. Test Coverage Criteria for RESTful Web APIs. A-TEST, 2019. doi:10.1145/3340433.3342822